ISLAMIC UNIVERSITY / ICT-4202 LABORATORY

Linear Regression, k-Nearest Neighbors & K-Means Clustering

A visual, mathematical and viva-focused study guide

Three complete topics. One practical revision pack.

Step-by-step formulas • solved mathematics • full-size diagrams
verified laboratory results • implementation anchors • viva answers

Linear Regression	Sets 2, 4, 8
k-NN Classification	Sets 1, 6, 10; WEKA IBk also in Sets 4 and 8
K-Means Clustering	Sets 3, 7, 9

Prepared using the laboratory question paper, your uploaded Python repository, and the AI/ML course references in this project. Numerical results labeled as laboratory outputs were reproduced from the actual provided scripts. Worked toy examples are separately labeled.

How to use this guide in the exam

VIVA ANSWER / KEY IDEA

For any experiment, answer in this order: **(1) problem type, (2) algorithm idea, (3) main equation, (4) what the code does, (5) output interpretation, (6) one limitation.** This works well for oral viva and for explaining the laboratory notebook.

At a glance: Do not confuse the three methods

Question	Regression	k-NN	K-Means
Learning type	Supervised	Supervised (classification here)	Unsupervised
Predict / find	Number	Class label	Cluster assignment
Needs class labels?	Yes, a numeric target	Yes	No
Main idea	Fit best straight line	Vote among nearest training points	Move centroids and assign points
Main output	Slope, intercept, predictions, R^2	Predictions, accuracy, confusion matrix	Centroids, inertia, silhouette
Meaning of k	Not applicable	Neighbor count	Cluster count

Fast reading order

1. **Linear Regression:** learn $\hat{y} = b_0 + b_1 x$, slope/intercept, MSE, R^2 and Set 2/4/8 outputs.
2. **k-NN:** learn Euclidean distance, majority voting, k selection, scaling and confusion matrix.
3. **K-Means:** learn centroid calculation, assignment/update, inertia, elbow and silhouette.
4. Finish with the **common viva questions and one-page formula sheet**.

Source distinction: All results marked *From your script* are from the uploaded code. Hand calculations and explanatory diagrams are added to help you understand the mathematics; they are not separate results from the question paper.

1. Linear Regression

1.1. What problem does it solve?

VIVA ANSWER / KEY IDEA

Linear Regression is supervised numerical prediction. It learns a straight-line relationship from labeled numerical examples. Example: use study hours (X) to predict examination score (Y). It predicts numbers, not class categories.

The main equation

$$\boxed{\hat{y} = b_0 + b_1 x} \tag{1}$$

x	Input or independent variable; e.g., study hours.
y	Actual or observed target; e.g., real exam score.
$\hat{y}$	Predicted output produced by the fitted line.
b_0	Intercept: predicted Y when $X = 0$ (if meaningful).
b_1	Slope: change in predicted Y when X increases by one unit.

Explain it like this: "If the fitted equation is $\hat{y} = 1 + 2x$, the intercept is 1, the slope is 2, and each extra unit of X increases the predicted Y by 2. At $x = 15$, the predicted Y is $1 + 2(15) = 31$."

What happens in code?

1. Prepare pairs of observed X and Y values.
2. Fit a straight line using `LinearRegression().fit(X, y)`.
3. Read `coef_` for the slope and `intercept_` for the intercept.
4. Use `predict(...)` for a new input and R^2 to evaluate fit.
5. Plot the observed dots and the fitted straight line.

1.2. How does the line get its slope and intercept?

Ordinary least squares chooses b_0 and b_1 to minimize the squared prediction errors.

$$\boxed{b_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}} \qquad \boxed{b_0 = \bar{y} - b_1 \bar{x}} \tag{2}$$

The equivalent computational slope formula is

$$b_1 = \frac{n\sum x_i y_i - (\sum x_i)(\sum y_i)}{n\sum x_i^2 - (\sum x_i)^2}.$$

1.3. Solved mathematics: actual Set 4 data

Set 4 supplies $X = (2, 4, 6, 8, 10, 12)$ and $Y = (5, 9, 13, 17, 21, 25)$.

x_i	y_i	x_i^2	$x_i y_i$
2	5	4	10
4	9	16	36
6	13	36	78
8	17	64	136
10	21	100	210
12	25	144	300
$\sum x = 42$	$\sum y = 90$	$\sum x^2 = 364$	$\sum xy = 770$

Step 1: Calculate slope.

$$b_1 = \frac{6(770) - 42(90)}{6(364) - 42^2} = \frac{4620 - 3780}{2184 - 1764} = \boxed{2}.$$

Step 2: Calculate intercept.

$$b_0 = \frac{90 - 2(42)}{6} = \boxed{1}.$$

Step 3: Write the fitted equation and predict.

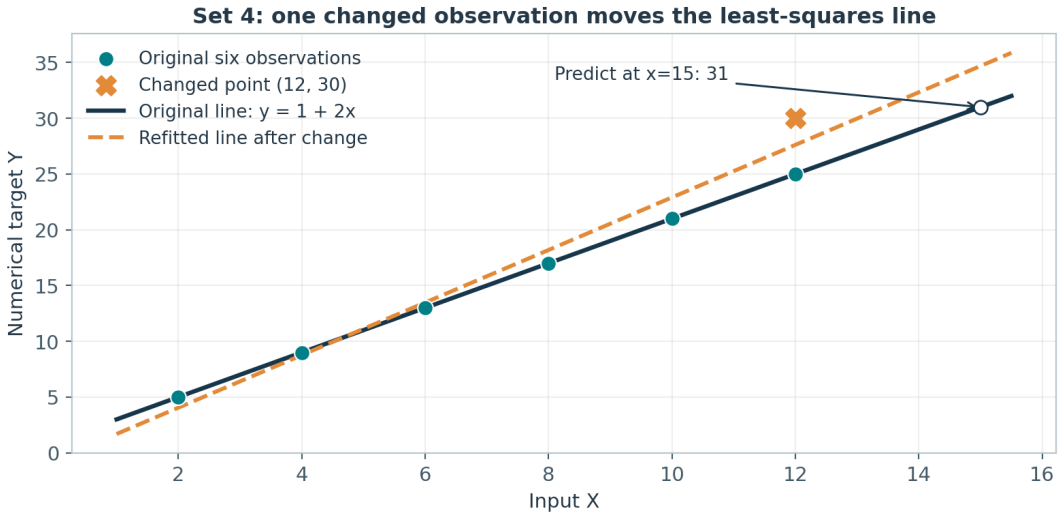
$$\boxed{\hat{y} = 1 + 2x}, \qquad \hat{y}(15) = 1 + 2 \times 15 = \boxed{31}.$$

VIVA ANSWER / KEY IDEA

Set 4 viva answer: "My original six points lie exactly on $\hat{y} = 1 + 2x$. The slope is 2, the intercept is 1, the prediction at $X = 15$ is 31, and the original R^2 is 1 because the residual error is zero."

1.4. Three concepts you must distinguish

- **Observed value** y**:** what the dataset actually contains.
- **Predicted value** $\hat{y}$**:** what the model calculates using its equation.
- **Residual** $e_i = y_i - \hat{y}_i$**:** actual minus predicted. It may be positive or negative.



Visual from the actual Set 4 input values: original fit and the refitted line after changing one target.

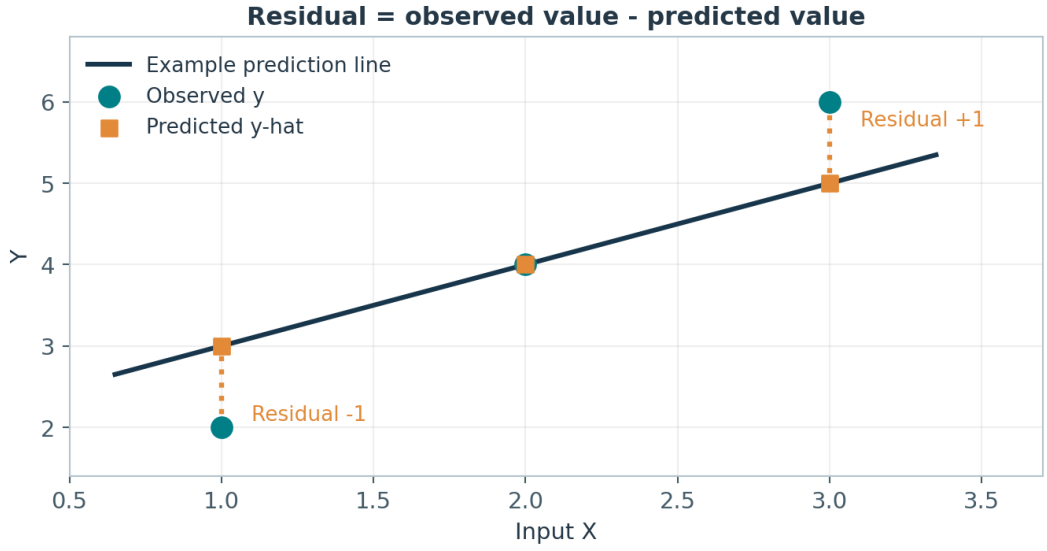
1.5. Cost function, MSE and R^2

Cost (loss) function: how wrong are the predictions?

For linear regression, a common cost function is Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{3}$$

The model chooses a line with small squared errors. Some teaching materials use $\frac{1}{2n}$ instead of $\frac{1}{n}$; this changes the scale of the function, not its minimizing line.



Each vertical gap is a residual. The sum of the squared gaps is the residual sum of squares (SSE).

Small MSE worked example

Actual values $(2, 4, 6)$; predictions $(3, 4, 5)$.

$$\text{SSE} = (2-3)^2 + (4-4)^2 + (6-5)^2 = 1 + 0 + 1 = 2.$$

$$\text{MSE} = \frac{\text{SSE}}{3} = \boxed{0.667} \text{ (approximately).}$$

1.6. Coefficient of determination: R^2

$$\boxed{R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}} = 1 - \frac{\text{SSE}}{\text{SST}}. \tag{4}$$

Here, SSE is the error of the model and SST is the error obtained when predicting the observed mean for everyone. For the same values $(2, 4, 6)$, the mean is $\bar{y} = 4$ and

$$\text{SST} = (2-4)^2 + (4-4)^2 + (6-4)^2 = 8.$$

Thus $R^2 = 1 - \frac{2}{8} = \boxed{0.75}$.

VIVA ANSWER / KEY IDEA

$R^2 = 0.75$: on these data, the model reduces squared error by 75% relative to the mean-only prediction. In a standard in-sample regression interpretation, it explains 75% of the observed variation. A value of 1 means perfect fit; 0 means no improvement over the mean; a negative value is also possible.

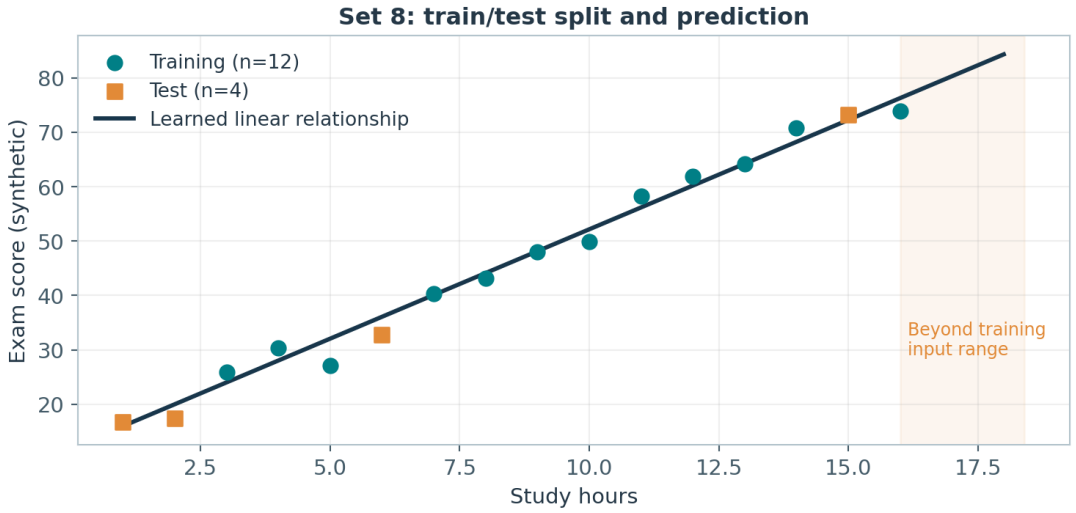
1.7. Your laboratory results: Sets 2, 4 and 8

The following values were reproduced by running your uploaded `Set_XX_C.py` scripts.

Set	What you did	What to report in viva
02	Fit 6 study-hours/scores pairs; then change the last score from 14 to 18 and refit.	Original: slope 1.9714, intercept 1.6000, prediction at 7 hours 15.4000, $R^2 = 0.9786$. Modified: slope 2.5429, intercept 0.2667, prediction 18.0667, $R^2 = 0.8922$.
04	Fit 6 perfectly collinear points; then change last Y from 25 to 30 (illustrative choice in the script).	Original $\hat{y} = 1 + 2x$, $\hat{y}(15) = 31$, $R^2 = 1$. Modified slope 2.3571, intercept -0.6667, $R^2 = 0.9703$.
08	Generate 16 synthetic study-hour observations; split 12 train / 4 test; fit and evaluate.	Slope 4.0211, intercept 11.9432. Train $R^2 = 0.9806$; test $R^2 = 0.9909$. Predicted at $x = 7.5$: 42.101; at $x = 18$: 84.323.

Set 8 caution: These are *synthetic*, generated scores, not actual student survey results. Predicting at 18 hours is extrapolation beyond the observed input range and may be

unreliable.



Actual Set 8 experiment: model fitted to 12 training observations and evaluated on four test observations.

Only five code expressions to recognize

Python expression	Meaning
`LinearRegression()`	Create the regression model.
`model.fit(X, y)`	Fit coefficients to observed data.
`model.coef_[0]` / `model.intercept_`	Read slope / intercept.
`model.predict([[7]])`	Predict an unseen input $X = 7$.
`model.score(X, y)`	Return R^2 for this regression.

1.8. Linear Regression: most likely viva

Q. Is regression a classification problem?

Answer: No. Here it predicts a continuous numerical target, not a category. It is a supervised regression task.

Q. Why is the equation a straight line?

Answer: Because the expected prediction has a linear dependence on one input: $\hat{y} = b_0 + b_1 x$.

Q. What is the slope?

Answer: The predicted change in Y for a one-unit increase in X. In Set 4 it is exactly 2.

Q. What is the intercept?

Answer: The model prediction at $X = 0$. It may not be practically meaningful if $X = 0$ is outside the relevant context.

Q. Why does least squares square the errors?

Answer: Squaring avoids positive/negative cancellation and penalizes large errors more heavily.

Q. What does $R^2 = 1$ mean?

Answer: Perfect fit on the evaluated observations: all residuals are zero. It does not guarantee future predictions will be perfect.

Q. Can R^2 become negative?

Answer: Yes. A model can predict worse than a constant mean baseline, especially on held-out test data.

Q. Why did Set 2 R^2 decrease after changing one score?

Answer: The changed value made a straight-line fit less adequate relative to the new outcome variation. Least squares refitted the coefficients.

Q. Training and test R^2: why both?

Answer: Training R^2 describes fit on training observations; test R^2 assesses performance on observations excluded from training.

Q. Are predictions the same as expected value?

Answer: A fitted regression estimate of $E[Y \mid X = x]$ is the conditional mean under the model assumptions. Individual observed values may differ due to noise or other factors.

Q. What is an arbitrary linear equation?

Answer: An equation such as $y = mx + c$ with coefficients not yet estimated. Regression uses data to select the fitted coefficients.

Q. What is one limitation?

Answer: A straight-line model may not represent nonlinear patterns; unusual or influential points can noticeably change the fit.

VIVA ANSWER / KEY IDEA

Say in 20 seconds: "I use Linear Regression for numerical prediction. I fit $\hat{y} = b_0 + b_1 x$ using least squares, read the slope and intercept, predict new values, and judge errors using MSE and model fit using R^2. I check the plot and whether a changed observation affects the fitted line."

2. k-Nearest Neighbors (k-NN)

2.1. What is k-NN and how does it work?

VIVA ANSWER / KEY IDEA

k-NN is a supervised, distance-based algorithm. For classification, it finds the k closest labeled training observations and predicts the most common class among them. In your laboratory, it predicts the Iris flower species.

Easy four-step algorithm

1. Choose the value of k (number of neighbors).
2. Calculate each training observation's distance from the new observation.
3. Select the k observations with the smallest distances.
4. Use a majority vote of their class labels to predict the new class.

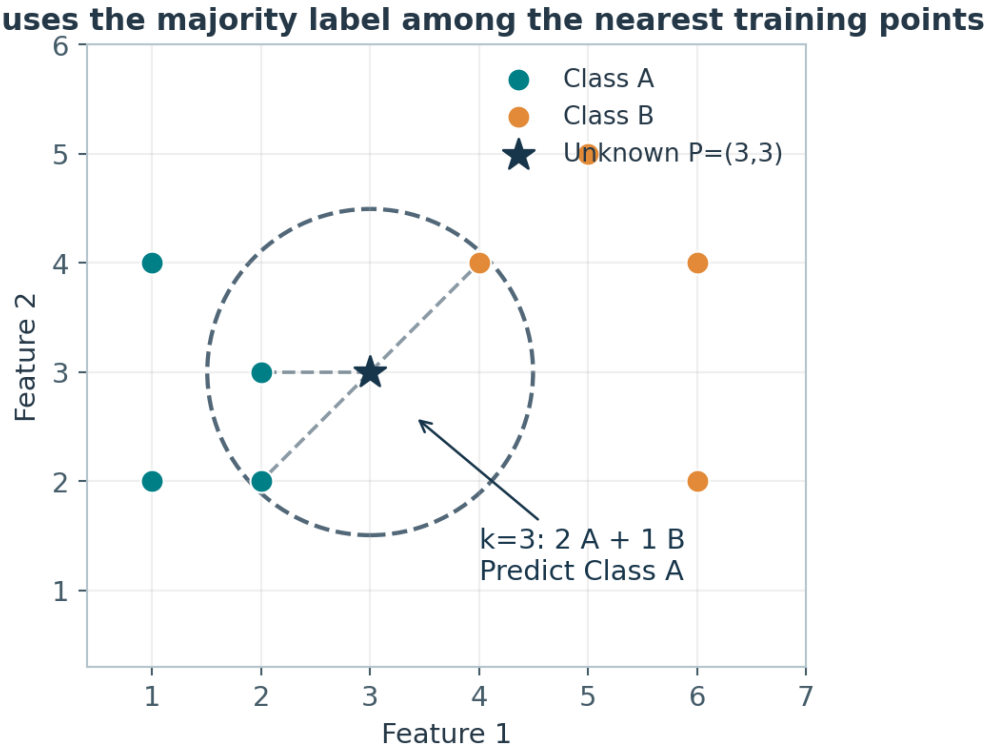
Euclidean distance is the essential equation

For two observations with p numerical features:

$$\boxed{d(\boldsymbol{x}, \boldsymbol{z}) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}} \tag{5}$$

For two dimensions $\boldsymbol{x} = (x_1, x_2)$ and $\boldsymbol{z} = (z_1, z_2)$:

$$d = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}.$$



Toy two-feature example, added for understanding. The three dashed nearest-neighbor links give 2 votes for A and 1 for B.

2.2. Worked distance and majority-vote calculation

A new point is $P = (3, 3)$; nearby training points include:

Point / label	Coordinates	Euclidean distance from P
A1 (Class A)	$(2, 3)$	$\sqrt{(3-2)^2 + (3-3)^2} = 1$
A2 (Class A)	$(2, 2)$	$\sqrt{1^2 + 1^2} = 1.414$
B1 (Class B)	$(4, 4)$	$\sqrt{(-1)^2 + (-1)^2} = 1.414$
A3 (Class A)	$(1, 2)$	$\sqrt{2^2 + 1^2} = 2.236$

For $k = 3$, nearest labels are A, A and B. Therefore, $\boxed{\text{prediction = Class A}}$.

2.3. How do we select k and why does scaling matter?

What happens if k changes?

- **Small k (e.g., 1):** follows local examples closely; can be sensitive to noise and outliers.
- **Larger k:** smoother voting; if too large, may ignore local patterns.
- **Select k:** compare candidates using validation or cross-validation, then evaluate once on an independent test set.

Why scale features?

k-NN calculates numerical distances. If one feature varies from 0 to 1 but another varies from 0 to 1000, the second may dominate the distance even if it is no more important.

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{6}$$

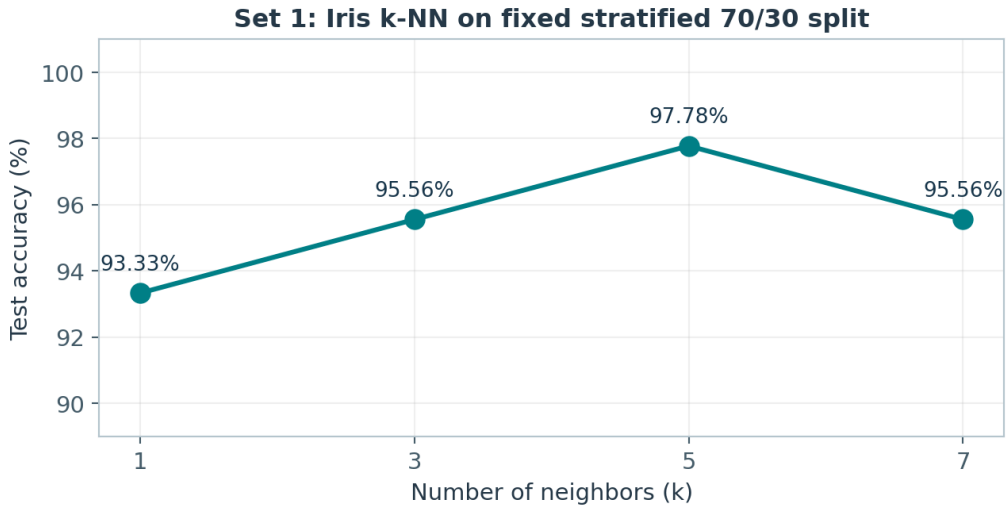
This is standardization: subtract the *training* mean and divide by the *training* standard deviation. A proper `Pipeline(StandardScaler(), k-NN)` prevents test-data leakage.

Important distinction: Scaling often matters for k-NN, but scaling does *not* automatically improve accuracy. Your Set 1 actual fixed-split accuracy decreased after scaling. That is one dataset/split result, not a universal rule.

2.4. Real k-NN results from Set 1

The code uses all four numerical Iris measurements (150 samples), a stratified 70/30 split, `random_state` $= 42$, and $k \in \{1, 3, 5, 7\}$.

k	Test accuracy
1	93.33%
3	95.56%
5	**97.78%**
7	95.56%



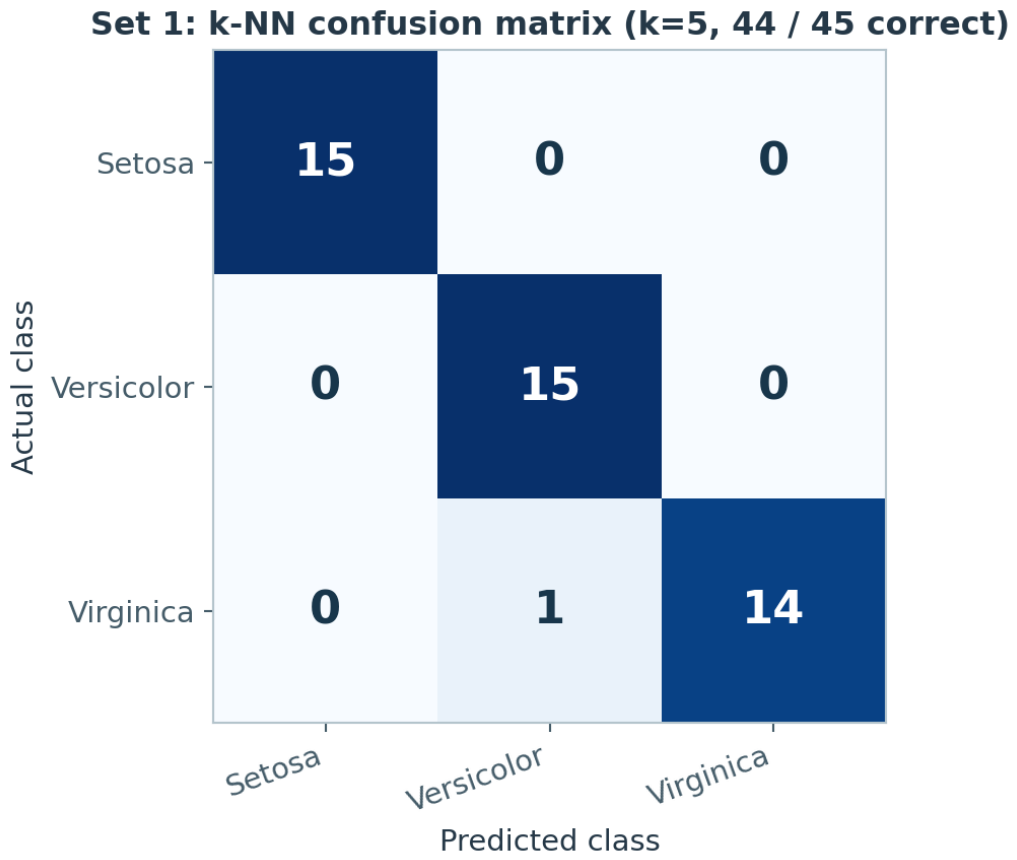
Actual Set 1 script output: best k among the tested values on this fixed split was 5.

After standardization with the same $k = 5$, test accuracy was **91.11%** (rather than unscaled 97.78%). **A subtle exam point**

Your Set 1 script chooses the best k based on the test split, to satisfy the laboratory comparison. In rigorous model evaluation, k should instead be selected on a validation split or with cross-validation, leaving the final test set untouched.

2.5. Confusion matrix, accuracy, precision, recall and F1

Your Set 1 k-NN with $k = 5$ gives the following actual test confusion matrix:



Rows are true class labels; columns are predicted labels. Of 45 test flowers, 44 were classified correctly.

Accuracy calculation

$$\boxed{\text{Accuracy} = \frac{\text{correct predictions}}{\text{all predictions}} = \frac{44}{45} = 97.78\%}.$$

For any **one selected class** in a binary or one-vs-rest interpretation:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}.$$

Read the confusion matrix aloud: "All 15 Setosa were correct; all 15 Versicolor were correct; 14 Virginica were correct and one Virginica was predicted as Versicolor. Therefore 44 of 45 predictions are correct."

Why not just use accuracy?

If one class is much more frequent than another, high accuracy can hide poor performance for the minority class. Confusion matrices, per-class precision, recall and F1 show what kinds of errors occurred.

2.6. Your k-NN laboratory implementations

Set	Required task / settings	Verified output / viva focus
01	Iris, 70/30 stratified; $k = 1, 3, 5, 7$; confusion matrix; standardization.	Highest observed accuracy is 97.78% at $k = 5$; scaled accuracy 91.11% for that k.
06	Iris: compare k-NN ($k = 5$) and Decision Tree (max_depth $= 3$); inspect errors; scale k-NN.	Both scored 97.78% on this split, but made different mistakes. Scaled k-NN scored 91.11%.
10	Iris: compare k-NN ($k = 3$) and Decision Tree (max_depth $= 3$); report precision/recall/F1.	k-NN 95.56% accuracy, macro F1 0.9554. Decision Tree 97.78%, macro F1 0.9778.
04 / 08	WEKA Explorer tasks: compare J48 decision tree and IBk nearest-neighbor classifier on Iris.	Use **WEKA itself** for actual 10-fold cross-validation, accuracy and confusion matrix. Python scripts only inspect the ARFF file.

2.7. WEKA's IBk is k-NN

1. Open WEKA Explorer, load `iris.arff` and verify the `class` target.
2. Go to **Classify**; select **lazy** → **IBk**.
3. Enter the neighbor count required in the set; choose **10-fold cross-validation** when instructed.
4. Start the classifier and read the **Correctly Classified Instances** and **Confusion Matrix**.
5. Compare with J48 under matching data/evaluation settings; save results for demonstration.

No invented WEKA accuracy: Your `Set_04_B.py` and `Set_08_B.py` confirm that Iris has 150 rows and three classes. They *do not* run WEKA IBk/J48. WEKA results must be recorded from the application.

Minimal Python anchors

Expression	What it means
`KNeighborsClassifier(n_neighbors=5)`	Select $k = 5$.
`train_test_split(..., stratify=y)`	Preserve class proportions in both sets.
`make_pipeline(StandardScaler(), ...)`	Scale within the learning workflow.
`confusion_matrix(y_test, pred)`	Show actual vs predicted classes.
`accuracy_score(y_test, pred)`	Calculate percentage of correct predictions.

2.8. k-NN: most likely viva

Q. Does k-NN need labels?

Answer: Yes. It is supervised for this Iris classification task, because neighboring training observations have known species labels.

Q. What does k mean?

Answer: The number of nearest training samples used to predict a new sample. It is not the number of classes.

Q. Is k-NN the same as K-Means?

Answer: No. k-NN predicts labels using labeled neighbors; K-Means groups unlabeled observations around cluster centroids.

Q. Why is k-NN called a lazy learner?

Answer: It retains the training examples rather than learning a compact global decision rule; most work occurs at prediction time.

Q. Which distance is used in your experiment?

Answer: The default Minkowski distance with power $p = 2$ in scikit-learn, equivalent to Euclidean distance.

Q. Why use an odd k?

Answer: In binary classification, odd k reduces simple voting ties; for three classes, ties are still possible.

Q. Does greater k always improve accuracy?

Answer: No. Too small may overfit noise; too large may overlook local class boundaries.

Q. Why standardize before k-NN?

Answer: Distance is affected by feature units; scaling can keep a high-range feature from dominating the distances.

Q. What is stratified splitting?

Answer: It approximately preserves the proportion of each class in the train and test sets.

Q. What is the Iris dataset?

Answer: A classic 150-observation classification dataset with three flower species and four measured numerical features.

Q. If k-NN and Decision Tree have equal accuracy, are they equivalent?

Answer: No. They can misclassify different individual cases, use different assumptions, and differ in speed and sensitivity to scale.

Q. How can you improve k-NN fairly?

Answer: Tune k and preprocessing with a validation set or cross-validation, avoid leakage, and evaluate once on independent held-out data.

VIVA ANSWER / KEY IDEA

Say in 20 seconds: "k-NN is supervised classification. I calculate distances from a new flower to labeled training flowers, choose the k closest, and predict the majority species. I compare test accuracy and confusion matrices. Since distance depends on feature scales, I also test standardization."

3. K-Means Clustering

3.1. What is K-Means and what does it predict?

VIVA ANSWER / KEY IDEA

K-Means is unsupervised clustering. It separates unlabeled numerical observations into k groups, aiming for small distances between points and their own cluster centroids. It finds groups; it does not use Iris species labels while fitting.

The algorithm in five simple steps

1. Choose the number of clusters k.
2. Select or initialize k centroids.
3. Assign each observation to the *nearest* centroid (usually Euclidean distance).
4. Recalculate each centroid as the *mean* of its assigned observations.
5. Repeat assignments and centroid updates until the clustering stabilizes or a stopping condition is met.

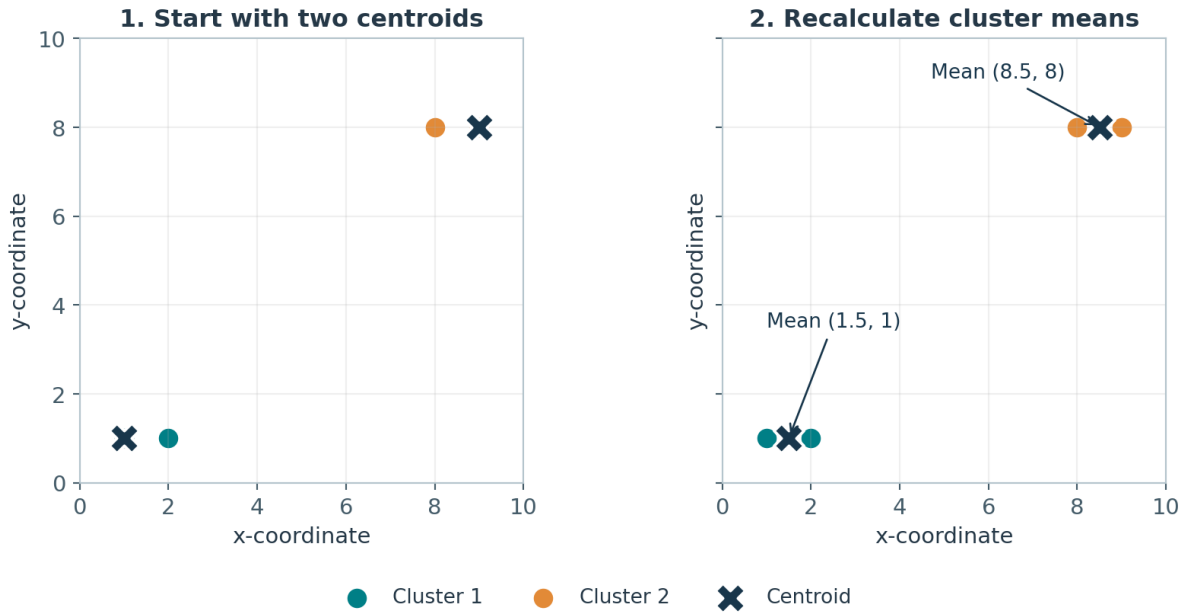
Centroid formula

For cluster C_j of size $|C_j|$:

$$\boxed{\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\boldsymbol{x}_i \in C_j} \boldsymbol{x}_i} \tag{7}$$

Distance used for assignment

$$d(\boldsymbol{x}_i, \boldsymbol{\mu}_j) = \sqrt{\sum_{r=1}^{p} (x_{ir} - \mu_{jr})^2}.$$



Toy calculation: after assignment, the two cluster means become (1.5,1) and (8.5,8).

3.2. Solved K-Means arithmetic: centroid and inertia

Use four *illustrative* points: $(1,1)$, $(2,1)$, $(8,8)$ and $(9,8)$. Let $k=2$. **Step 1. Assign points to their closest center**

With initial centers $(1,1)$ and $(9,8)$, the first two points belong to Cluster A, the last two to Cluster B. **Step 2. Update both centroids**

$$\boldsymbol{\mu}_A = \left(\frac{1+2}{2}, \frac{1+1}{2}\right) = \boxed{(1.5, 1)}.$$

$$\boldsymbol{\mu}_B = \left(\frac{8+9}{2}, \frac{8+8}{2}\right) = \boxed{(8.5, 8)}.$$

Step 3. Calculate the K-Means objective: inertia

Inertia is the *within-cluster sum of squared distances*:

$$\boxed{J = \sum_{i=1}^{n} \left\| \boldsymbol{x}_i - \boldsymbol{\mu}_{c_i} \right\|^2} \tag{8}$$

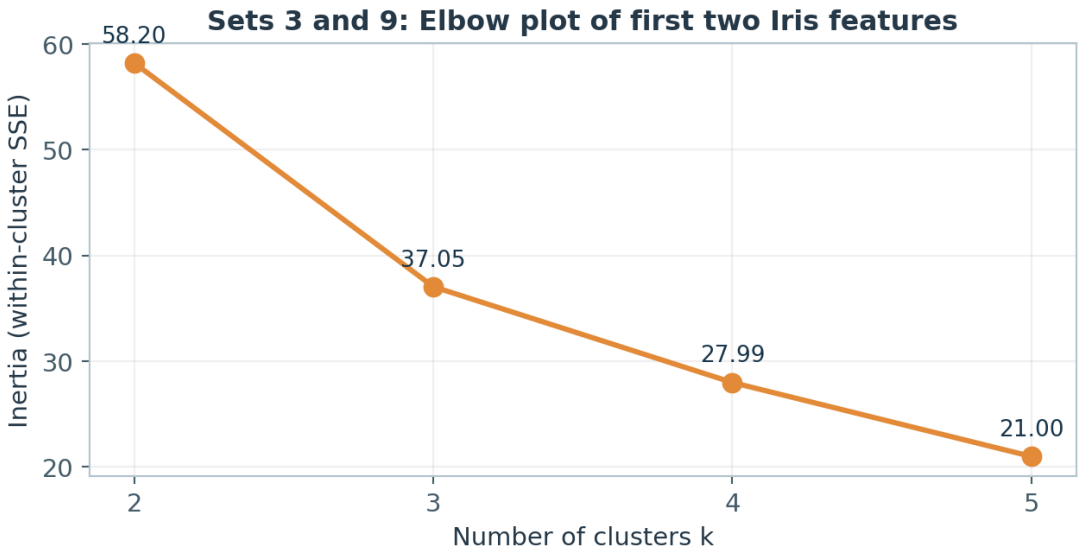
For our four points:

$$J = 0.25 + 0.25 + 0.25 + 0.25 = \boxed{1.0}.$$

Simple viva response: "The centroid is the average of all points in a cluster. Inertia measures the total squared distance from each point to its assigned centroid. K-Means alternates between assigning observations and recalculating centroids to lower this objective."

3.3. How to choose k? The elbow method

Run clustering for different k values and plot inertia against k. As k increases, inertia cannot increase for a global optimum and normally decreases for practical runs. Look for an **elbow**, where adding more clusters yields diminishing reductions in inertia.



Actual Sets 3 and 9 inertia results on two Iris features. The elbow may not be sharply defined.

Why not choose the smallest inertia automatically? The smallest inertia is usually achieved with more clusters; if each distinct point had its own centroid, the training inertia could be zero. Therefore, inertia alone does not determine the best meaningful k.

3.4. Silhouette: the second method for choosing k

For one observation i:

- $a(i)$ is average distance from point i to points in its own cluster.
- $b(i)$ is its smallest average distance to any other cluster.

$$\boxed{s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}} \tag{9}$$

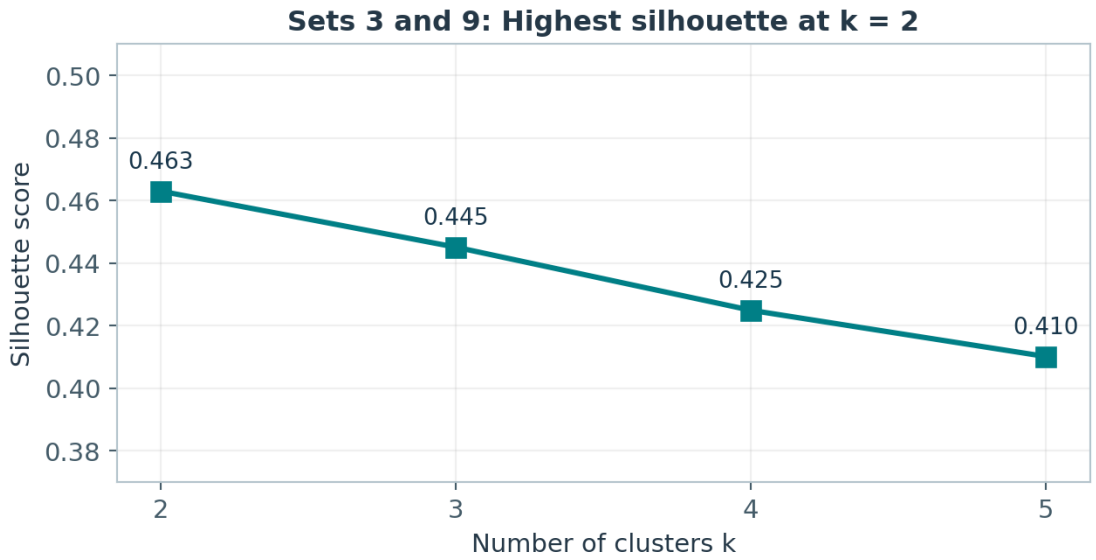
Score	Interpretation
Close to $+1$	Point is close to its own cluster and far from other clusters.
Near 0	Point is on or near the boundary between clusters.
Below 0	Point may be closer on average to another cluster.

Quick worked example

If $a(i) = 2$ and $b(i) = 6$, then

$$s(i) = \frac{6-2}{\max(2,6)} = \frac{4}{6} = \boxed{0.667}.$$

The *overall silhouette score* is the average across points (when valid for the cluster configuration).



Actual Sets 3 and 9: $k = 2$ has the highest mean silhouette among the tested values 2,3,4,5.

VIVA ANSWER / KEY IDEA

Exam decision: "Using the first two Iris features in Sets 3 and 9, the largest silhouette among $k = 2, 3, 4, 5$ occurs at $k = 2$. This is a model-selection result for this feature space; it does not have to match the three known Iris species."

3.5. Sets 3 and 9: model selection, centers and output

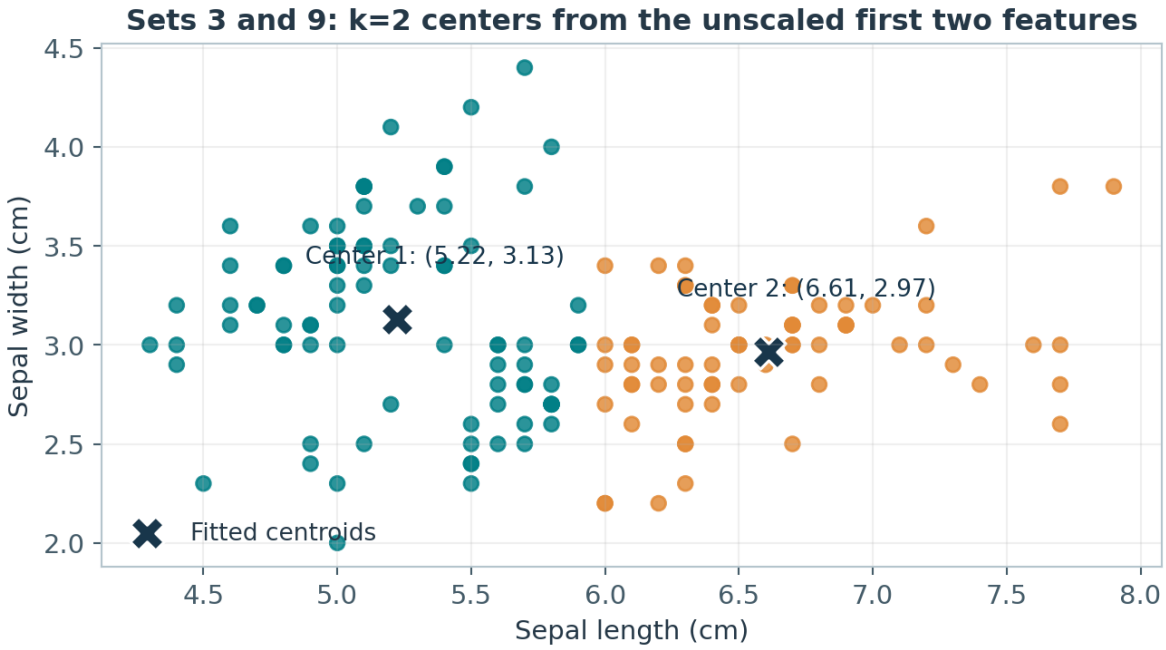
Both scripts use the first two Iris features only: **sepal length** and **sepal width**, unscaled, and test $k = 2, 3, 4, 5$ with `n_init=10`, `random_state=42`.

k	Inertia	Silhouette
2	**58.2041**	**0.4630**
3	37.0507	0.4451
4	27.9902	0.4249
5	20.9968	0.4101

Centers for the selected $k = 2$ model

$$\boldsymbol{\mu}_1 \approx (5.224, 3.131), \qquad \boldsymbol{\mu}_2 \approx (6.610, 2.966).$$

Coordinates are (sepal length, sepal width), in centimeters. Cluster IDs and the order of centroids are arbitrary and can switch without changing the clustering.



Actual selected K-Means clustering for Sets 3/9, with large X markers indicating the centroids.

3.6. What exactly does the code do?

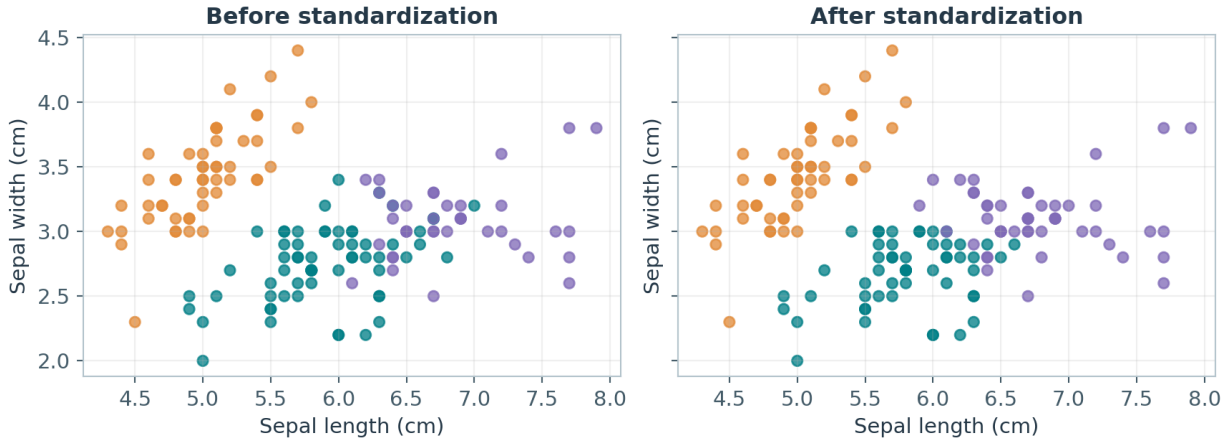
1. `X = iris.data[:, :2]` chooses the first two measured features.
2. `KMeans(n_clusters=k, n_init=10, ...)` fits each candidate cluster count.
3. `model.labels_` stores a cluster ID for each sample.
4. `model.cluster_centers_` stores the final cluster means.
5. `model.inertia_` returns the total within-cluster squared distance.
6. `silhouette_score(X, model.labels_)` evaluates average silhouette.

3.7. Set 7: why scaling changes K-Means

Unlike Sets 3 and 9, Set 7 fits K-Means with **all four numerical Iris features**, $k = 3$. It compares unscaled measurements in centimeters against standardized z-scores.

Feature space	Inertia	Silhouette
All four Iris features, original cm	78.8514	0.5528
Four standardized features, z-scores	139.8205	0.4599

Set 7: cluster membership can change after feature scaling



Both clusterings were fit on all four features; the diagrams display only sepal length/width to visualize the membership differences.

Critical viva explanation: The unscaled and standardized inertia numbers are expressed in *different coordinate systems*. Do not say that inertia 78.85 is automatically better than 139.82. Scaling changes distances, centroids, assignments and sometimes silhouettes.

How do centroids change units?

- Unscaled centroids are in original Iris feature units (cm).
- Scaled centroids are in standardized units (z-scores).
- To interpret scaled centroids in cm, use `scaler.inverse_transform(centers)`.

K-Means and k-NN: a comparison you must know

	k-NN	K-Means
Method	Supervised classifier	Unsupervised clusterer
k	Number of neighbors	Number of clusters
Reference	Labeled data points	Centroids (means)
Assignment	Majority voting	Nearest centroid
Output	Predicted class	Cluster membership
Evaluation	Accuracy / confusion matrix	Inertia / silhouette

3.8. K-Means: most likely viva

Q. Is K-Means supervised or unsupervised?

Answer: Unsupervised: it does not use target class labels while finding groups.

Q. What is a centroid?

Answer: The component-wise arithmetic mean of points assigned to a cluster.

Q. How is the nearest centroid chosen?

Answer: Compare distances from a point to all centroids and select the smallest distance.

Q. What is inertia?

Answer: The sum of squared distances from observations to the centroids of their assigned clusters.

Q. Why does inertia decrease as k increases?

Answer: More centers give additional flexibility to place each observation near some center; the minimum possible within-cluster SSE cannot increase.

Q. Which method selects k?

Answer: The elbow method uses decreasing inertia; the silhouette method looks for high cohesion and separation. Domain meaning can also guide the decision.

Q. What is a silhouette score?

Answer: A measure of how much closer an observation is to its own cluster compared with another cluster; higher is generally better.

Q. Why was $k = 2$ chosen in Sets 3 and 9?

Answer: Its silhouette of 0.4630 was the largest among the tested values 2, 3, 4 and 5.

Q. But Iris has three species; why only two clusters?

Answer: Species labels are not used by K-Means. A two-dimensional feature projection can naturally form a different number of compact groups.

Q. How can scaling change the result?

Answer: It changes distances and the relative influence of features, which changes centroids and cluster membership.

Q. Do the centroids remain the same after scaling?

Answer: Not necessarily. They are computed in transformed space; inverse-transform them when reporting values in cm.

Q. What is one limitation of K-Means?

Answer: It needs a chosen k, can be sensitive to scaling/initialization/outliers, and favors relatively compact clusters under Euclidean distance.

4. One-page final revision

The seven formulas to memorize

$$\begin{aligned}
\text{Regression line:} \quad & \hat{y} = b_0 + b_1 x \\
\text{Slope:} \quad & b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \\
\text{Intercept:} \quad & b_0 = \bar{y} - b_1 \bar{x} \\
\text{MSE:} \quad & \frac{1}{n} \sum (y_i - \hat{y}_i)^2 \\
\text{Coefficient of determination:} \quad & R^2 = 1 - \frac{\text{SSE}}{\text{SST}} \\
\text{Euclidean distance:} \quad & d(\boldsymbol{x}, \boldsymbol{z}) = \sqrt{\sum_j (x_j - z_j)^2} \\
\text{K-Means centroid:} \quad & \boldsymbol{\mu}_c = \frac{1}{|C|} \sum_{x_i \in C} \boldsymbol{x}_i
\end{aligned}$$

Two clustering equations you may also be asked

$$\text{Inertia} = \sum_i \|\boldsymbol{x}_i - \boldsymbol{\mu}_{c_i}\|^2, \qquad s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

Don’t lose marks on these four traps

1. **Not every k means the same thing:** k-NN means neighbors, K-Means means clusters.
2. **High R^2 is not classification accuracy:** R^2 is for regression performance.
3. **A smaller K-Means inertia across different scales is not necessarily better:** units differ.
4. **Scaling does not guarantee higher accuracy or silhouette:** report your measured output.

Five fastest answers to give an examiner

1. “Regression predicts a number by fitting $\hat{y} = b_0 + b_1 x$.”
2. “The slope measures predicted change in Y for each one-unit increase in X.”
3. “k-NN predicts a class using votes from nearby labeled samples.”
4. “K-Means groups unlabeled points using repeatedly updated centroids.”
5. “A confusion matrix is for classification; R^2 is for regression; inertia and silhouette are for clustering.”

Sources, scope and exam conditions

Main sources (uploaded in this conversation and project):

- *AI_MLQuestionList(3).pdf*, ICT-4202 Final Laboratory Examination, Sets 1–10.
- *ICT-4202-AI-ML-Laboratory-main(1).zip*, the Question 3 (C) Python scripts for Sets 01, 02, 03, 04, 06, 07, 08, 09 and 10.
- Course references: *08ClassBasic.pdf*, *ICT-4201_classifier.pdf*, *ICT-4201_clustering.pdf*, and *10ClusBasic.pdf* (classification and clustering foundations).
- Three students' oral viva reports supplied in the conversation (used to emphasize common oral questions such as R^2, cost, centroid, inertia and choice of k).

Verification note: Numerical results above are from running the uploaded laboratory code with the fixed values and seeds in those scripts. Results from other code or package versions may differ slightly. This document does not claim to have executed WEKA GUI algorithms; consult WEKA's own output for its accuracy and confusion matrices.

Exam habit: When the examiner points to any chart, first identify the X-axis, Y-axis, model or experiment, then explain one concrete number and one reason it changed. For code, identify the one line that performs the main algorithmic operation.

Good luck with your laboratory viva.